IBM Cloud API Key Security — GitHub & Project

Why this matters

Exposed API keys can result in data breaches, unexpected cloud spend, service disruption, and audit or certification failures. Protecting credentials is a mandatory professional practice.

What to avoid

- Do not hard-code API keys or secrets directly in source code.
- Do not commit API keys to GitHub (public or private repositories).
- Do not share API keys via chat tools, email, or screenshots.

Recommended approach (step-by-step)

Step 1 — Store keys in environment variables

Use environment variables to keep secrets out of source code and reduce accidental exposure in version control.

- **Decide the variable name**: for IBM Cloud automation, use **IBMCLOUD_API_KEY** (common in scripts/REST examples). Some tools also accept **IC_API_KEY** (for example, certain IBM Cloud developer tooling). Standardize on one name across your project.
- **Set it on your machine** (local development):
  - **macOS / Linux (Terminal)**: **export IBMCLOUD_API_KEY="your_key_here"** (current session). To make it persistent, add the same line to **~/.zshrc** or **~/.bashrc** and restart the terminal.
  - **Windows PowerShell**: **$env:IBMCLOUD_API_KEY="your_key_here"** (current session). For a persistent user-level variable, use Windows “Environment Variables” settings.
- **Verify it is set**:
  - macOS / Linux: **echo $IBMCLOUD_API_KEY**
  - Windows PowerShell: **echo $env:IBMCLOUD_API_KEY**
- **Run your app from the same terminal** where the variable is available (common beginner issue).

- **Good practice**: keep separate keys for *dev* and *prod*, and rotate them periodically.

- **Set it on your machine** (local development):
  - **macOS / Linux (Terminal)**: **export IBMCLOUD_API_KEY="your_key_here"** (current session). To make it persistent, add the same line to **~/.zshrc** or **~/.bashrc** and restart the terminal.
  - **Windows PowerShell**: **$env:IBMCLOUD_API_KEY="your_key_here"** (current session). For a persistent user-level variable, use Windows "Environment Variables" settings.
- **Verify it is set**:
  - macOS / Linux: **echo $IBMCLOUD_API_KEY**
  - Windows PowerShell: **echo $env:IBMCLOUD_API_KEY**
- **Run your app from the same terminal** where the variable is available (common beginner issue).
- **Good practice**: keep separate keys for *dev* and *prod*, and rotate them periodically.

Step 2 — Use a *.env* file (beginner-friendly)

1. Create a *.env* file in your project root.
1. Add your IBM Cloud API key, for example: **IBMCLOUD_API_KEY=your_key_here**
2. Add *.env* to *.gitignore* to prevent commits.
3. **Recommended file format**: one key per line (no quotes needed unless your value contains spaces). Example:
   - **IBMCLOUD_API_KEY=your_key_here**
4. **Do not add extra spaces** around **=** (e.g., avoid **IBMCLOUD_API_KEY = ...**).
5. **Create a safe template for the team**: add a non-secret **.env.example** file with placeholder values so others know what to set.
6. **Load .env in your app** (only for local/dev):
   - Python: install **python-dotenv**, then load it at startup using **load_dotenv()**.
   - Node.js: install **dotenv**, then call **require('dotenv').config()** (CommonJS) or **import 'dotenv/config'** (ESM).

7. **Sanity check**: print a masked value during debugging (e.g., first 4 characters only). Never print the full key in logs.

2. Add your IBM Cloud API key, for example:
   IBMCLOUD_API_KEY=your_key_here

3. Add *.env* to *.gitignore* to prevent commits.
4. **Recommended file format**: one key per line (no quotes needed unless your value contains spaces). Example:

Step 3 — Read the key in your application code

- Python: **os.getenv('IBMCLOUD_API_KEY')**
- JavaScript / Node.js: **process.env.IBMCLOUD_API_KEY**
- **Add a startup check**: fail fast with a clear error message if **IBMCLOUD_API_KEY** is missing (example: "IBMCLOUD_API_KEY not set").
- **Keep config in one place**: read the environment variable once (e.g., in a config file/module) and pass it to the SDK/client.
- **Common learner issues**:
    - Variable name mismatch (e.g., using **IBMCLOUD_APIKEY** in one place and **IBMCLOUD_API_KEY** in another).
    - App started from an IDE that does not inherit your terminal environment—set environment variables in your IDE run configuration if needed.
    - Using the wrong key for the wrong environment (dev vs prod).
- **Never** store the key in client-side code (browser/mobile apps). Use a backend to keep keys private.
- JavaScript / Node.js: **process.env.IBMCLOUD_API_KEY**
- **Add a startup check**: fail fast with a clear error message if **IBMCLOUD_API_KEY** is missing (example: "IBMCLOUD_API_KEY not set").
    - Variable name mismatch (e.g., using **IBMCLOUD_APIKEY** in one place and **IBMCLOUD_API_KEY** in another).

For CLI-based workflows, log in using the environment variable (non-interactive):
ibmcloud login --apikey "$IBMCLOUD_API_KEY" -q --no-region

Step 4 — Using IBM Cloud API keys safely with GitHub

- **Use GitHub Secrets (recommended)**: store the key in your repository or organization secrets (Settings → Secrets and variables → Actions). Name it **IBMCLOUD_API_KEY**.
- **Reference the secret in GitHub Actions**: pass it as an environment variable, never echo it to logs.
- **Example (GitHub Actions)**:
  - Set: **env: IBMCLOUD_API_KEY: ${{ secrets.IBMCLOUD_API_KEY }}**
  - Login step: **ibmcloud login --apikey "$IBMCLOUD_API_KEY" -q --no-region**
- **Block accidental commits**: ensure *.env*, credential files, and build artifacts are ignored in *.gitignore*.
- **Scan before you push**: run **git status**, **git diff --staged**, and **git grep -n "IBMCLOUD_API_KEY\|apikey\|token\|secret"**.
- **Enable secret scanning**: turn on GitHub Advanced Security (if available) or use a pre-commit secret scanning tool so leaks are caught early.

Step 5 (production) — Use IBM Cloud Secrets Manager

- **Store the API key centrally** in an IBM Cloud Secrets Manager instance (avoid distributing long-lived keys across developer machines).
- **Control access** with secret groups and IAM policies (least privilege).
- **Retrieve at runtime**: applications and pipelines fetch the secret when needed; the key stays out of source code and GitHub.
- **Rotate regularly** and update deployments automatically after rotation.

If a key is exposed

1. **Contain immediately (stop the leak)**: treat the key as compromised as soon as you suspect exposure.
   - If it is in a repo, assume it has been copied—do not wait for confirmation.
   - Do not share the key further (no screenshots, no chat pastes).
2. **Revoke the IBM Cloud API key** (IAM): go to **IBM Cloud console → Manage → Access (IAM) → API keys**, then **delete** (preferred) or **disable** the exposed key.
   - If possible, use a **Service ID** key for automation instead of a personal user key.
   - Keep access minimal: grant only the roles required for the task.

3. **Rotate and remediate everywhere** (GitHub + CI/CD + local):
    - **Create a new key** (preferably for a service ID) and validate it works.
    - **Update GitHub Secrets**: set **IBMCLOUD_API_KEY** under **Settings → Secrets and variables → Actions** (repo/org), and rerun the workflow.
    - **Update other locations**: CI/CD tools, deployment platforms, and local *.env* files.
    - **Remove the key from Git history** if it was committed (even briefly). After cleaning, force-push only if approved by your team and you understand the impact on collaborators.
    - **Check for suspicious activity**:
    - Review recent usage and billing/spend anomalies for affected services.
    - Audit recent deployments and workflow runs that may have used the exposed key.
    - **Prevent recurrence**:
    - Move secrets to **GitHub Secrets** (and **IBM Cloud Secrets Manager** for production) instead of files or code.
    - Enable secret scanning / pre-commit scanning so commits are blocked when they contain keys.
    - Add a team rule: never log secrets; mask them in CI logs.